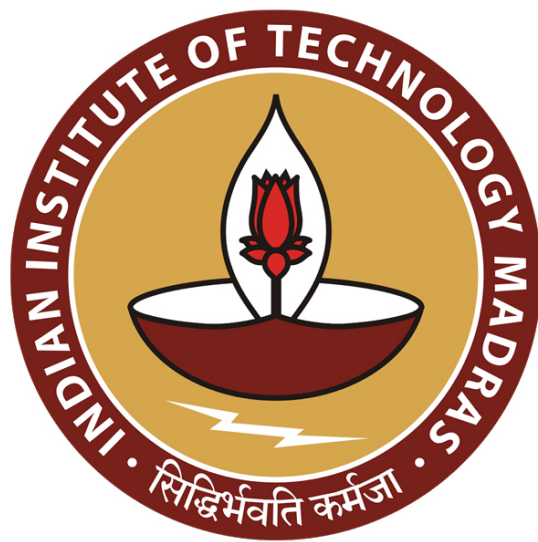




IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Introduction to Relational Model/1

Welcome to week 2, module 6 of Database Management System course in IIT, Madras online BSc program.

(Refer Slide Time: 0:38)

Week Recap

- The proliferation of DBMS in wide range of applications provide motivation for the subject
- Know Your Course provided information about prerequisites, outline and objectives
- The specific need for a DBMS discussed in contrast to a file system based application using a programming language like Python
- Basic notions of a DBMS are introduced

In the last week, the week 1, we have taken you through the introductory part of the course, giving you the course outline and basically sensitizing you on the proliferation of database management systems and the wide range of applications and specific need of database in contrast to the power that a file-based application can give us, why would we prefer a database system and when would you prefer a database system. And then in the last two modules 4 and 5, we talked about the basic notions of database. Now, slowly we will start getting into more specifics and that will be around the relational database system.

(Refer Slide Time: 1:21)

Module Objectives

- To understand attributes and their types
- To understand the mathematical structure of relational model
 - Schema
 - Instance
 - Keys
- To familiarize with different types of relational query languages

So, in this module, we will try to understand the attributes, their roles and talk about the basic mathematical structure of the relational model in terms of schema, instance and keys and familiarize with very basics of relational query languages.

(Refer Slide Time: 1:40)

Module Outline

- Attribute Types
- Relation Schema and Instance
- Keys
- Relational Query Languages

These will be the module components, which you can see on the left.

(Refer Slide Time: 1:46)

Example of a Relation

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000

So, this example of a relation, this I think you have already got quite accustomed to because you have seen it a number of times particularly this example or its variant. So, it is a table a relation is nothing but a table, which has columns identified by names. So, basically the order of the columns do not again matter, but the names do matter. So, these are called attributes. And then you have according to the columns, you have multiple rows, every row is a record or a row or a tuple, which has specific values on those attributes. So, this is the, this is a basic form of a relation.

(Refer Slide Time: 2:28)

Attributes

Attributes

Attribute Types

- Consider Students = Roll#, First Name, Last Name, DoB, Passport#, Aadhaar relation
- The set of allowed values for each attribute is called the **domain** of the attribute
 - Roll #: Alphanumeric string
 - First Name, Last Name: Alpha String
 - DoB: Date
 - Passport #: String (Letter followed by 7 digits) – nullable (optional)
 - Aadhaar #: 12-digit number
 - Department: Alpha String
- Attribute values are (normally) required to be **atomic**; that is, indivisible

So, in this the first thing that comes is we need to know about what is an attribute. So, let us say we are talking about a relation or you want to keep the information of an entity called student so, that we would be able to have complete information about students. So, we say that so, we say that these are the different attributes, that is how do you identify a student or how do you describe a student. A person, student is a person, say person have millions of attributes, starting from the name and the parents name and address and hobbies and so on.

But certainly, when we try to model we will attribute, choose only those parameters of the entity or only those parameters have a student which are relevant for our purpose of application, that is a business decision. So, naturally, the student will have a roll number, that's the first thing required for enrolment. The first and last name which is very typical, the date of birth will be required, we can have we have passport number, but every student may not have a passport, Aadhaar number has become mandatory in India.

So, the student will have another number otherwise, some of the student benefits may not be available to the student and the student will always belong to a department, it is a very minimum and that there will be a lot of other also that will come in. So, these different attributes, each attribute I say is it is an atomic one. So, it has an atomic type, what type. So, what is the roll number, we take roll number is a alphanumeric string that it will have integers as well as it will have characters.

Now, you could have said that why not just a number, but this will become clear because through the roll number we want to also do some other hidden communication. First name and last name are naturally stream of alphas, alphabet. Date of birth is a datatype which we take as a atomic type, though it has got three components, but those three components are

represented in a way whether it is we say mm-dd-yy that is month two digits, dd is date two digits and yy is year two digits.

Or it could be yyyy mmm dd 4 digits for year, 3 spaces for month which could write month as Jan, Feb, Mar like that and 2 places for the digit. So, whatever it is this is this is kind of another atomic primitive type used. Passport is a string say, letter followed by 7 digits which is the standard followed in India. It is nullable. This is a important. Nullable means that if a person does not have a passport, how do I entered the data of that student? I cannot.

So, what we allow is we may just put a value called null and what the null means there is no value it is not known. So, not everything is nullable you cannot have null as your roll number you are not say I do not know the roll number, not possible or first name I do not have a first name, not possible. Then, you have Aadhaar number which all of us know is a 12-digit number. Department is alpha string.

So, this in this way every attribute will have certain types and the important thing is they are atomic they are indivisible that you cannot compose first name and last name together into a single attribute because that is not one alpha string that is a pair of alpha strings which is a composite value. And null we already have mentioned. Now let us go to see more of this.

(Refer Slide Time: 6:47)

Attribute Types

- For *Students* = Roll#, First Name, Last Name, DoB, Passport#, Aadhaar#
- And domain of the attributes as:
 - Roll #: Alphanumeric string
 - First Name, Last Name: Alpha String
 - DoB: Date
 - Passport #: String (Letter followed by 7 digits) – nullable (optional)
 - Aadhaar #: 12-digit number
 - Department: Alpha String

Roll #	First Name	Last Name	DoB	Passport #	Aadhaar #	D
15CS10026	Lalit	Dubey	27-Mar-1997	4032464	1728-6174-9239	Cc

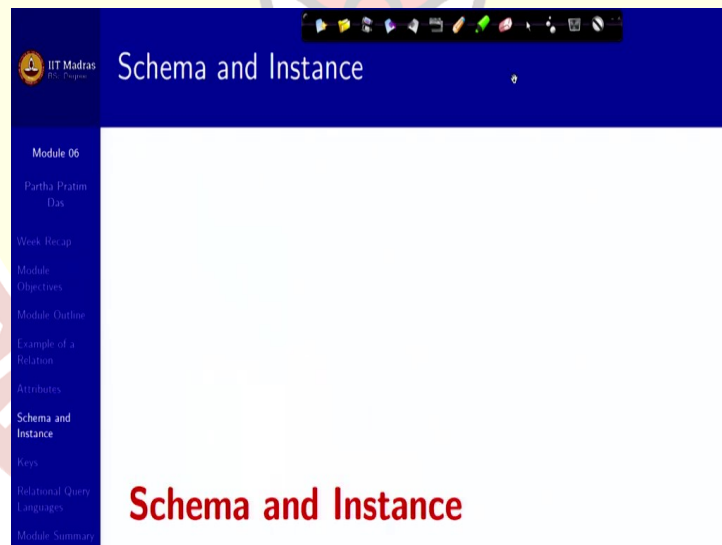
So again, the same. Now, the domain of every attribute, as we have specified means that if roll number is an alphanumeric string, what does it say? It says that any string composed of the A-to-Z alphabet and 0 through 9 numerals can be a possible value. So, you can see examples here. The first name is alpha. So, the domain is any string that you can form with

uppercase and lowercase alpha. The same thing here. In roll number you could specify that actually you could further qualify saying that is alphanumeric with the numbers and uppercase alphabet only.

But in name, you will have that. DOB is as it says primitive. The passport number in the format is null. So, Jatin Chopra does not have a passport. Here is a Aadhaar number. The dash demarcations are not part of the actual domain representation. I am still kept that because otherwise as human being it becomes difficult to read, but in the system and this will just be a 12-digit number and the name of the department which is alpha.

So, this is exactly given the, so here for example, in this DOB field, I cannot put a string I cannot say 27th March, 1997 I cannot write this because the type is date. Similarly, in here in terms of computer, I cannot write it is CSE235 like that. So, where 235 may happen to be the code, the code of the department, those kinds of things will have to be compatible with the domain, the values will have to be compatible with the domain. That is a basic property of the attributes.

(Refer Slide Time: 8:55)



Relation Schema and Instance

- $A_1, A_2, \dots, A_n$ are attributes
- $R = (A_1, A_2, \dots, A_n)$ is a relation schema
Example: instructor = (ID, name, dept_name, salary)
- Formally, given sets $D_1, D_2, \dots, D_n$ a **relation** r is a subset of

$$r \subseteq R \subseteq D_1 \times D_2 \times \dots \times D_n$$

Thus, a relation is a set of n -tuples $(a_1, a_2, \dots, a_n)$ where each $a_i \in D_i$

- The current values (**relation instance**) of a relation are specified by
- An element t of r is a tuple, represented by a row in a table
- Example:

Now, next let us talk about schema and instance. We mentioned it little earlier. Schema is what the attributes, in relation what attributes are and now we know that every attribute has an associated type. So, schema talks only about that. Instance talks about the actual values that the tuples, the rows take according to the schema.

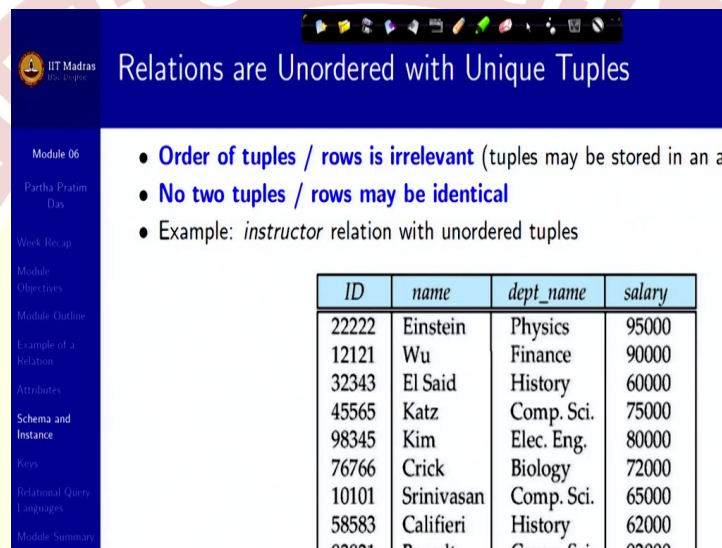
So, in the schema is representable mathematically as a relation, I mentioned that you need to know the concept of relation and Cartesian product and all that. So, I hope you have revised if not, this is the last time you will have to revise those. So, if there are n attributes, then we say a relation $R(A_1, A_2, \dots, A_n)$ is a relational schema because it says that these n attributes are there. So, what does it mean?

If I have an instructive with 4 attributes, then this is the relational schema of the instructor. So, now if we say D_1, D_2, D_n are domains of the respective attributes, so A_1 has a domain which is D_1 that is for the attribute A_1 given the type I know what are the universal set of values that can happen. So, any value for that attribute A_1 has to come from this D_1 set that is the basic point.

Then, the relation is a subset of so, R is basically a subset of $D_1, D_2; D_1 \text{ cross } D_2 \dots \text{cross } D_n$ because you are making an n tuple out of n different sets D_1 to D_n . So, any component of R or a particular row small r belonging to this relation will have values a in A_1 which is a member of D_1 , value for A_2 which is a member of D_2 , a value for an which is a member of D_n that is the basic. So, that r will be at an n tuple A_1, A_2, A_n written in small case where each A_i belongs to D_i This is a basic structure of the relational schema.

The relational instance or the instance would be the table of actual set of values. So, for example of relational schema here it say that instructor is actually is, is a schema of string 5, by string 5 I mean string of length 5, because IDs have to be a 5 length. Cross string, which is the name of the instructor, cross string again, which is the name of the department, cross number plus; number plus means one or more numbers in the sequence, which means it is an numerical value which is salary. So, that is the basic the correspondents. I think this is easy to understand.

(Refer Slide Time: 12:11)



Relations are Unordered with Unique Tuples

- **Order of tuples / rows is irrelevant** (tuples may be stored in an arbitrary order)
- **No two tuples / rows may be identical**
- Example: *instructor* relation with unordered tuples

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
82821	Bass	Comp. Sci.	82000

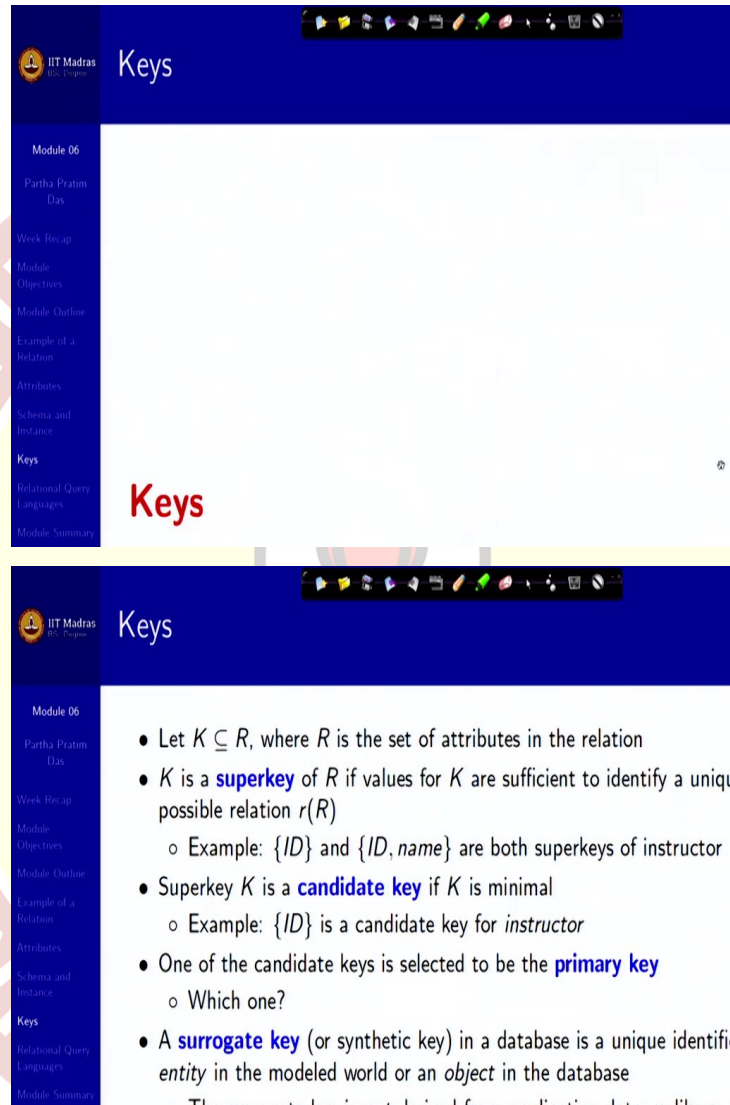
So, this is how now, this is the instance that I have is given these columns and that we are not again talking about their types. So, the columns and their domains have given us the schema and according to that schema, I will now have a relational instance which is a set of tuples, set of values that we have.

Now, two things is are very important to remember always is the order of tuples in the table, in the relation is irrelevant like 22222 Einstein etcetera as a first row I can put it anywhere else, as long as the total set of rows are same, the two relation instances are same, the order of rules does not matter.

Because you have to remember if you recall, a relation is nothing but a set and a set has no ordering between its member elements. So, every row is a member element of the relation set. So, it has no ordering expectation. At the same time, the other specification of the set says that every element has to be distinct.

So, which means that there can be no two rows or tuples which are identical. Every tuple will have to be distinct from every other tuples. So, there will have to be at least one field where this table would vary in the value for all other tuples.

(Refer Slide Time: 13:51)



Keys

- Let $K \subseteq R$, where R is the set of attributes in the relation
- K is a **superkey** of R if values for K are sufficient to identify a unique possible relation $r(R)$
 - Example: $\{ID\}$ and $\{ID, name\}$ are both superkeys of *instructor*
- Superkey K is a **candidate key** if K is minimal
 - Example: $\{ID\}$ is a candidate key for *instructor*
- One of the candidate keys is selected to be the **primary key**
 - Which one?
- A **surrogate key** (or synthetic key) in a database is a unique identifier *entity* in the modeled world or an *object* in the database
 - The surrogate key is not derived from application data, unlike a natural key

So, this is, so let us talk about keys next. Key, the basic concept of the keys there are multiple attributes. Now, not all of them uniquely identify a row they may, but some, the question is of minimality. Now naturally, since I cannot have two rows with identical set of values, so clearly all attributes taken together, my every row or every tuple must be unique, identifiable, but the question is, do I really need to know all the values to uniquely identify every row or can I work with a subset of attributes which can uniquely identify every row?

So, we say that let us assume that there is such a subset called k . So, k is a subset of R the relational schema, which is a set of attributes in the relation. Now, we say k is a super key of R if the values of k are sufficient to identify a unique tuple in each possible relation. Now, given a schema that can be actually potentially countably infinite number of relational instances, each one in each one of them I am saying that if I know the value on k , I will be able to uniquely identify that record. What is the concept say?

If I say the instructor has an ID, employee ID, we say if I know the ID, naturally two employees cannot have an identical ID they can have identical name, they can have identical department, they can have identical designation, they can have identical salary, they can have you know, so and so forth.

But they cannot have identical employee code. Two students cannot have identical role number. Two Indians cannot have identical Aadhaar number and so on so forth. So, I can say that ID is a super key of instructor, but anything which is bigger than ID say ID and Name will also be a superkey.

So, that is a question that is the reason you call this is a superkey, because you can say that all attributes they are also a key because they uniquely identify every row, but they are a key, but they are not formally called the key they will be called a superkey unless we can prove that there is no subset smaller than this, which can also serve that purpose. So, you said that is superkey is a candidate key, if k is minimal, what is minimality? Minimality means that k is a candidate key.

And no subset of k is it superkey, k is a superkey. And no subset of k is a superkey, then we will say it is a minimal set or it is a candidate key. So, both ID and ID name are superkeys. Of that, ID, ID name pair is just a superkey it is not a candidate, why? Because there is a subset containing just ID, which is also a superkey.

So, it is not minimal. So, ID and Name is a superkey, but ID the set containing ID is a candidate key, because, there is no subset because a subset of a single element will be obviously empty that is trivially true. But formally saying there is no subset of the set containing ID, which can be a superkey of this relation. So, it is a, this is a candidate key.

Now, it is not necessary that I will have only one candidate key, I may have multiple candidate keys, I will not use all of them, I cannot because does not make sense. So, of the candidate keys, I will choose one to be the primary key. And that is what we will use as the

key. And in the next slide, I will give you some glimpse of if there are multiple candidate keys, what is your, could be the factors to choose a particular one as a primary key.

Besides that, there are there could be surrogate or synthetic key in many cases that you actually the data here what is happening, you are saying that the data itself is unique. So, if the data itself may not be unique, for example, I am making an order on Big Bazaar. So, I can make multiple orders, I can make multiple orders of the same item, I can make multiple orders of the same item on the same quantity, I can make multiple orders of the same item of the same quantity on the same day and so on.

So, just looking at the data, it may not be possible or easy to uniquely find them, but the orders are different, they are not the same order they cannot be treated as the same, they may have the same set of items, same quantity of items, same price, they may have been ordered by the same person, they are going to the same address everything remaining same they are different orders. So, what do you do?

So, you synthetically generate a key that is order number. Order number is not there in the nature. You synthetically generated that and those synthetically generated data which are done at the time of transaction only are known as surrogate key. Now, you will ask that well, if I am saying it like this, then now, why not about why not you say that employee ID or roll number is also surrogate?

Way, one way they are, but they are not in the sense that if you think about the order, the order number is has a life only for that order. And as long as the order is served beyond that, you are not going to do anything with that order. So, in the collection of orders, it will remain uniquely identified, but the basic life cycle of an order is limited. It is initiated, it is transacted, paid, delivered closed. And after that you do not expect any changes to happen.

So, that is an entity which has a very short-lived time, which kind of relates to just a meta, meta transaction in the sense that set of transactions that are required to complete an order starting from the initiation to the delivery and closure. In contrast, the student or the instructor has a permanence in the system. The roll number remains valid not for one transaction or one meta transaction, it remains valid as long as the person is there.

So, in a way, roll number or ID are also synthetically created, but they uniquely identify a unique entity in the world. Whereas, an order number is synthetically created to uniquely find

and otherwise indistinguishable entity like an order. That is the difference. So, order number becomes a surrogate key or a synthetic key.

(Refer Slide Time: 22:02)

Keys

- **Students** = Roll #, First Name, Last Name, DoB, Passport #, Aadhaar #
- **Super Key:** Roll #, {Roll #, DoB}
- **Candidate Keys:** Roll #, {First Name, Last Name}, Aadhaar #
 - Passport # cannot be a key. Why?
 - Null values are allowed for Passport # (a student may not have a passport)
- **Primary Key:** Roll #
 - Can Aadhaar # be a key?
 - It may suffice for unique identification. But Roll # may have additional information. For example: 14CS92P01
 - ▷ Read 14CS92P01 as 14-CS-92-P-01
 - ▷ 14: Admission in 2014
 - ▷ CS: Department - CS

Now, an example. So, coming to example here is the student these attributes. So, what is the superkey, roll number is a superkey. Roll number and DOB of course is a superkey. DOB by itself will not be a superkey. So, what are the candidate keys? These are candidate keys, of course. Roll number.

Now passport number, you will ask why not passport number. Passport number is not a candidate key. Because it is not a super key because it is nullable which means that is optional. So, some students may not have a passport number. So, obviously I cannot use passport number.

If it is mandated that everyone will have a passport number, like every traveller on a flight on an international flight can be uniquely identified also by their passport number because they must have one. But this is not the case here. So, passport cannot be used. It is nullable. We can have roll number, we can have first name, last name pair and we can have Aadhaar number.

Of that again, so these are the candidate keys. So, we will have to make a choice for the primary key. So, how do I make that choice I said that the choice has to have additional design criteria. The one criteria here is if I take the first name last name, and say that this is a candidate key and want to make it a primary key, then it becomes evident that I cannot have two students with identical name.

I was, I have tried to look for Partha Pratim Das and Google even in Google Scholar, forget about the entire Google just in Google Scholar probably has 18 Partha Pratim Das. So, you can say that by first name and last name, it is not proper to identify students, because you will soon get into difficulties. So, this gets ruled out.

So, the choice remains between rule number and Aadhaar number, Aadhaar number you can obviously use because Aadhaar numbers again like a ID which has been created for your entire lifetime. But between the roll number and Aadhaar number in this application, we will prefer to use roll number because let us say and this specific example is certainly the way IIT Kharagpur does it or the other academic institutions will just do something similar.

So, this is a sample roll number of one of my students. Now, you have to read this in this format 14 dash CS dash 92 dash P dash 01. So, what does that mean? The 14 stands for year of admission, CS stands for the department code, 92 stands for the category of student what, whether is a doctoral student, whether is a Master student, whether it is a Masters by research student, is a undergraduate student, is a BSc program student, the MSC program student has different category rules.

Then, this letter says what is the type of admission is a person a regular student in undergraduate or the person or the student is actually getting some stipend from the Institute or the person is actually working in the institute or the person is working in a project in the institute and so on gives you a lot of those information.

And finally, given all this being same, the last two digits are for serial number, place is assume that in one category that not be more than more than 100, or more than 99 people have that. So, what happens is two things one is, it well the Aadhaar number could also serve the purpose, but the, here in terms of making the rule number as a primary key, we are doing some, shorthand encoding.

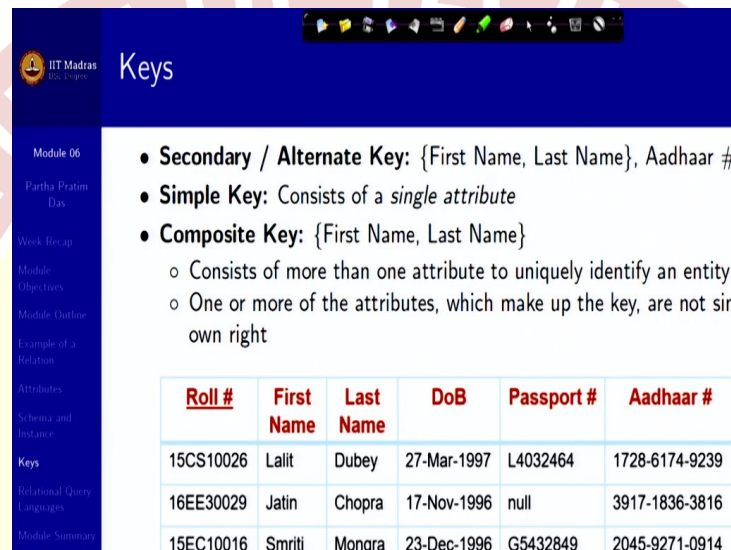
So, all of these are parameters, or are attributes of the person so, we could blow this up and keep them into separate components. Instead of doing that these have been compacted and encoded because very often you do not need to know these attributes separately. So, if you do, then you can always do a small algorithm to recover them.

So, this kind of, kind of, we are doing a trick where actually the key has a composition in motion, semantically it is composite, but syntactically it is an atomic value, an alphanumeric string. So, this composite idea of these, 1, 2, 3, 4, 5 six different things, I am sorry, five

different things have been put together to create a key, which is unique, and which tells you a lot more than just the uniqueness, if you use Aadhaar, you will not have this advantage.

So, these are some of the additional considerations by which the first name last name, and the Aadhaar number are not considered to be primary key from the candidate key and the roll number is. So, in every type of system you have, you may have these kinds of considerations to decide on that key.

(Refer Slide Time: 27:15)



Keys

- **Secondary / Alternate Key:** {First Name, Last Name}, Aadhaar #
- **Simple Key:** Consists of a *single attribute*
- **Composite Key:** {First Name, Last Name}
 - Consists of more than one attribute to uniquely identify an entity
 - One or more of the attributes, which make up the key, are not in their own right

Roll #	First Name	Last Name	DoB	Passport #	Aadhaar #
15CS10026	Lalit	Dubey	27-Mar-1997	L4032464	1728-6174-9239
16EE30029	Jatin	Chopra	17-Nov-1996	null	3917-1836-3816
15EC10016	Smriti	Mongra	23-Dec-1996	G5432849	2045-9271-0914

So, these are the different, so those candidate keys which are not primary are called secondary or alternate key. A simple key is a single attribute, like Aadhaar number, or roll number composite key is, which has more than one component, first name, last name, and based on that I have just shown a table here of possible values are a relational instance that you could have.

(Refer Slide Time: 27:46)

Keys

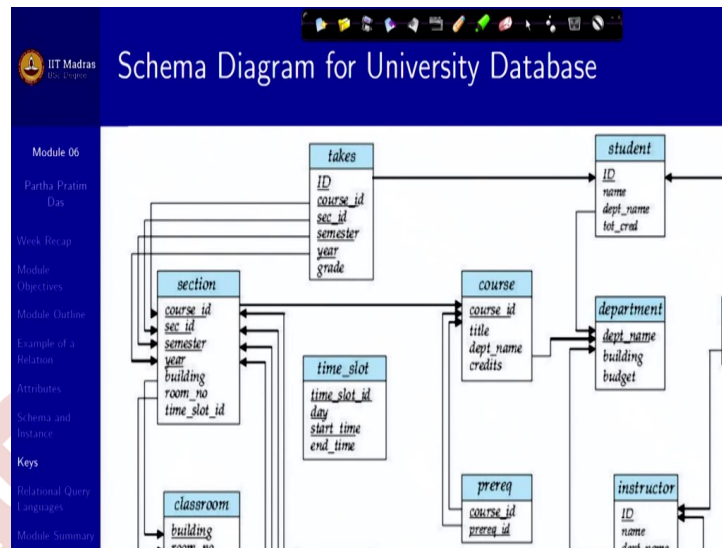
- **Foreign key constraint:** Value in one relation must appear in another
 - **Referencing** relation
 - ▷ Enrolment: Foreign Keys – Roll #, Course #
 - **Referenced** relation
 - ▷ Students, Courses
- A **compound key** consists of *more than one attribute* to uniquely identify an occurrence
 - Each attribute, which makes up the key, is a simple key in its own right
 - {Roll #, Course #}

Students						
Roll #	First Name	Last Name	DoB	Passport #	Aadhaar #	Department

There is also a concept of a foreign key. A foreign key is, and we will slowly understand this more is when a particular attribute is a key in a different table. So, you have students table, you can, you have to just look here, you have students table, which has roll number as the key, you have the courses table, where the course number is the key, and you have an enrolment table, which relates the student and the course, which student is in which course.

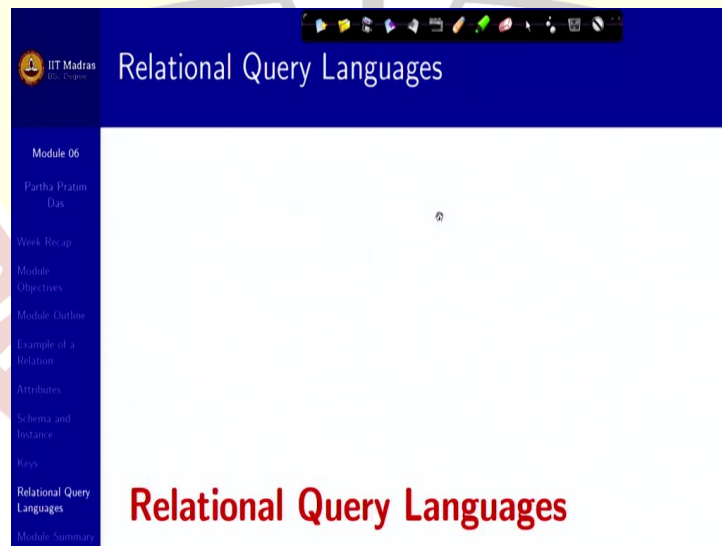
So, for the students table, Roll number is used as a key part of key in another table. So, this is it, this is what is known as a foreign key. So, somebody somewhere else, I am a key. So, it is very, very important. Now, incidentally, it is also a primary key in students, but it may always not be the case, something which is not a key in a relation could be a foreign key for some other relation.

(Refer Slide Time: 28:45)



So, this is I will not go through in in steps. This is just a little bit of more detailed schema design for the university database that I have given here. And I leave it to you to study and practice and understand and if you have questions to raise it in the forum, so that we will discuss, but this is a part of exercise that you have to do.

(Refer Slide Time: 29:07)





Relational Query Languages

Module 06
Partha Pratim Das
Week Recap
Module Objectives
Module Outline
Example of a Relation
Attributes
Schema and Instance
Keys
Relational Query Languages
Module Summary

Procedural viz-a-viz Non-procedural or Declarative Paradigms

- **Procedural programming** requires that the programmer tell the computer
 - That is, *how* to get the output for the range of required inputs
 - The programmer must know an appropriate algorithm
- **Declarative programming** requires a more descriptive style
 - The programmer must know *what* relationships hold between variables

Now, before we close, I like to just mention that what are the relational query languages because we have talked about the basic schema and keys and also how do you finally do the query. So, there are basically two ways that we can do a programming, one is called procedural, which tells you how you find a value. Most of the programs you write in C, C++ plus plus, Java, Python, they are all procedural. And the other is declarative, where you do not say, what, how you find the answer, but you say what you want as an answer.

(Refer Slide Time: 29:47)



Relational Query Languages (Cont..)

Module 06
Partha Pratim Das
Week Recap
Module Objectives
Module Outline
Example of a Relation
Attributes
Schema and Instance
Keys
Relational Query Languages
Module Summary

Procedural vs. Non-procedural or Declarative Paradigms

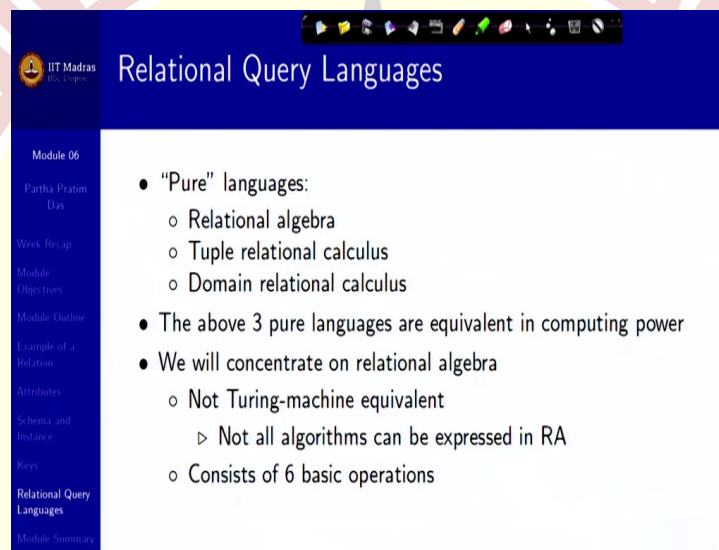
- **Example: Square root of n**
 - Procedural
 - a) Guess x_0 (close to root of n)
 - b) $i \leftarrow 0$
 - c) $x_{i+1} \leftarrow (x_i + n/x_i)/2$
 - d) Repeat Step 2 if $|x_{i+1} - x_i| > \text{delta}$
 - Declarative
 - ▷ Root of n is m such that $m^2 = n$

This example will help us. Think about finding the square root of a number. And this is one way there is a very standard known algorithm for that. You guess something which is close to the square root of n and then keep on iterating by doing refinements and checking the deviation of values and when the deviation becomes low, you have what the square root.

This whole set of steps if you see tells you how to get to the square, but it does not tell you that this is the square, you have to come to some external mathematics to prove that, well this is a square root if you do this you will get the square root.

So, 'what' is not known, but 'how' is known how I will do the computation is known this is procedure. Instead, if I could have a language and there are languages like this, where I just say that I want given n I want to find an m such that m squared is equal to n , then it is declarative because I am saying what I want not how I can get it. So, this is the basic difference.

(Refer Slide Time: 30:53)

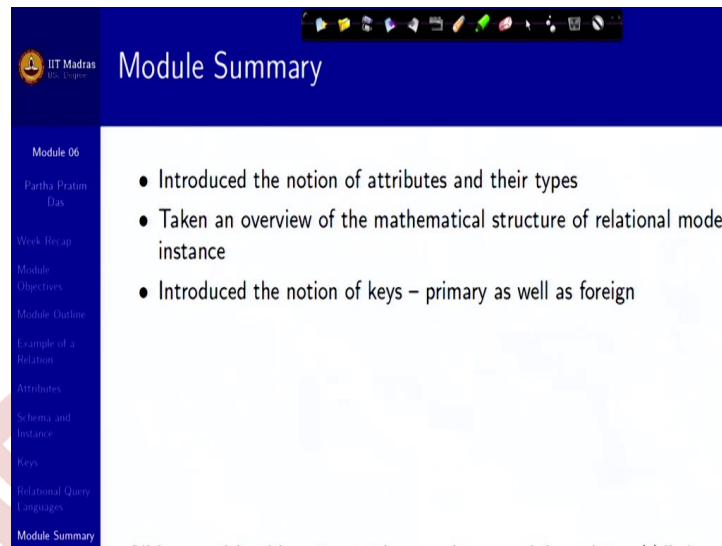


Relational Query Languages

- "Pure" languages:
 - Relational algebra
 - Tuple relational calculus
 - Domain relational calculus
- The above 3 pure languages are equivalent in computing power
- We will concentrate on relational algebra
 - Not Turing-machine equivalent
 - ▷ Not all algorithms can be expressed in RA
 - Consists of 6 basic operations

And in terms of that, we will see that it is the declarative style which is useful for relational query and there are three models I have already talked of relational algebra, relational tuple calculus and relational domain calculus and also remember that they are not turing complete.

(Refer Slide Time: 31:18)



Module Summary

- Introduced the notion of attributes and their types
- Taken an overview of the mathematical structure of relational model instance
- Introduced the notion of keys – primary as well as foreign

Module 06
Partha Pratim Das
Week Recap
Module Objectives
Module Outline
Example of a Relation
Attributes
Schema and Instance
Keys
Relational Query Languages
Module Summary

So with this, I, we come to the close of this module where we got started in the relational model and discussed about the basic notions. We will continue this in the next module. Thank you for your attention and see you again in module 7.